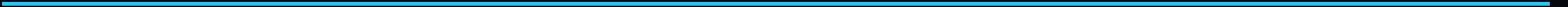


Classificação da Qualidade de Vinhos com Machine Learning

Glaucyely Soares Guilherme

Objetivo

- Físico-químicas
- Existe uma forma de predizermos a qualidade de um vinho?



Dataset Wine Quality

- 1143 amostras
- 11 variáveis físico-químicas

```
df.shape  
  
(1143, 13)
```

```
df.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

Análise Exploratória (EDA)

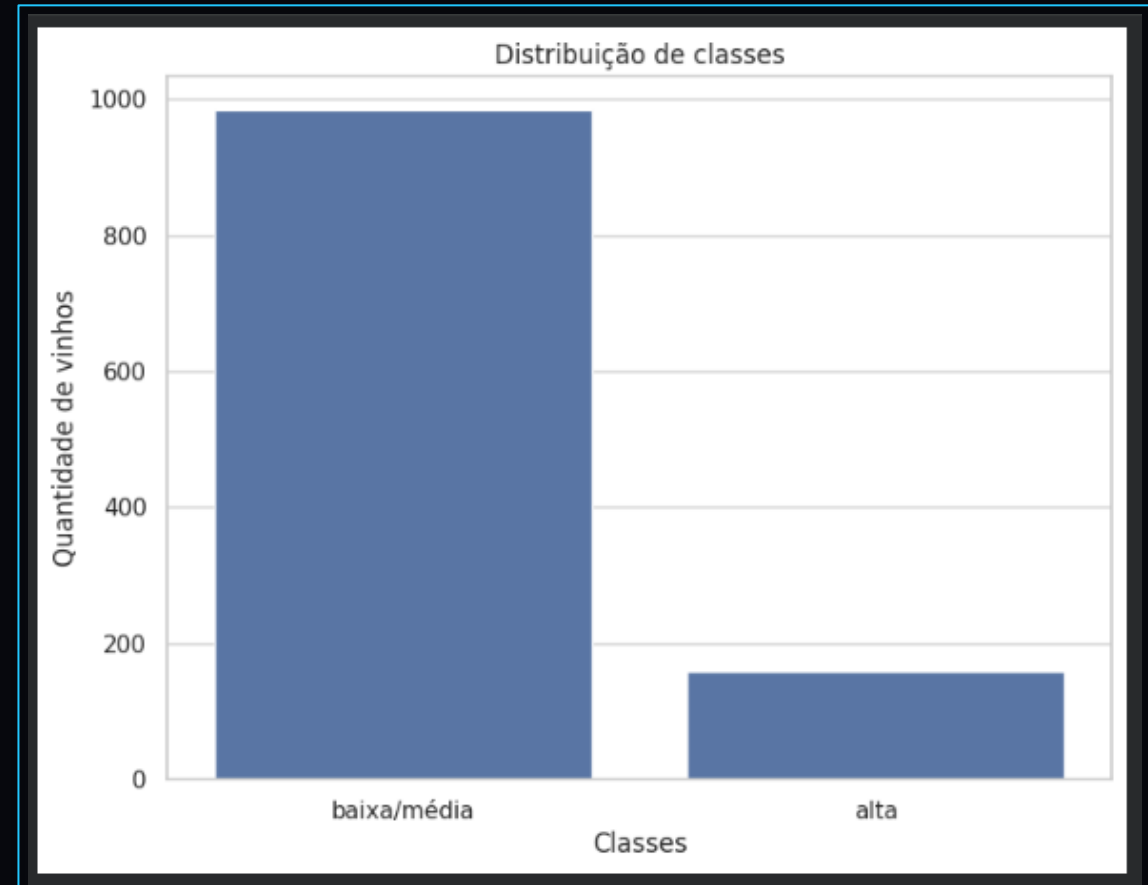
- Distribuição da qualidade em classificação binária

Alta qualidade ≥ 7

Baixa/média qualidade < 7

count	
quality_binary	
0	984
1	159

Divisão em números binários



Distribuição da qualidade

Análise Exploratória (EDA)

- Mapa de correlação
- Qualidade
- Teor Alcólico (Correlação Positiva)
- Acidez Volátil (Correlação Negativa)

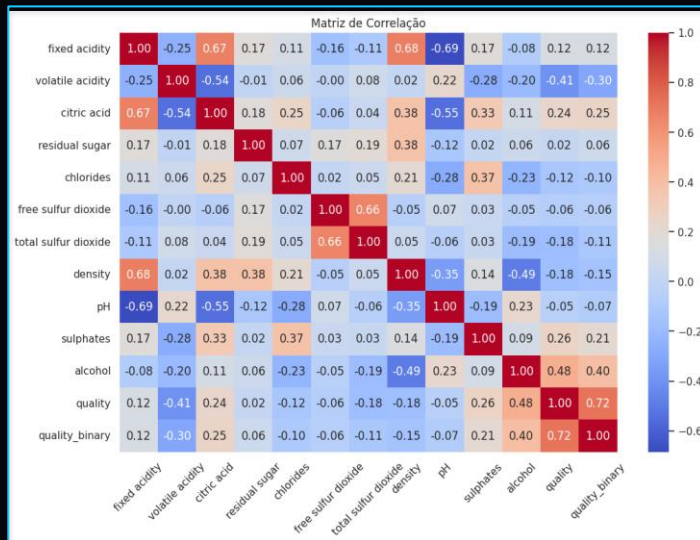


Gráfico de correlação em Hitmap

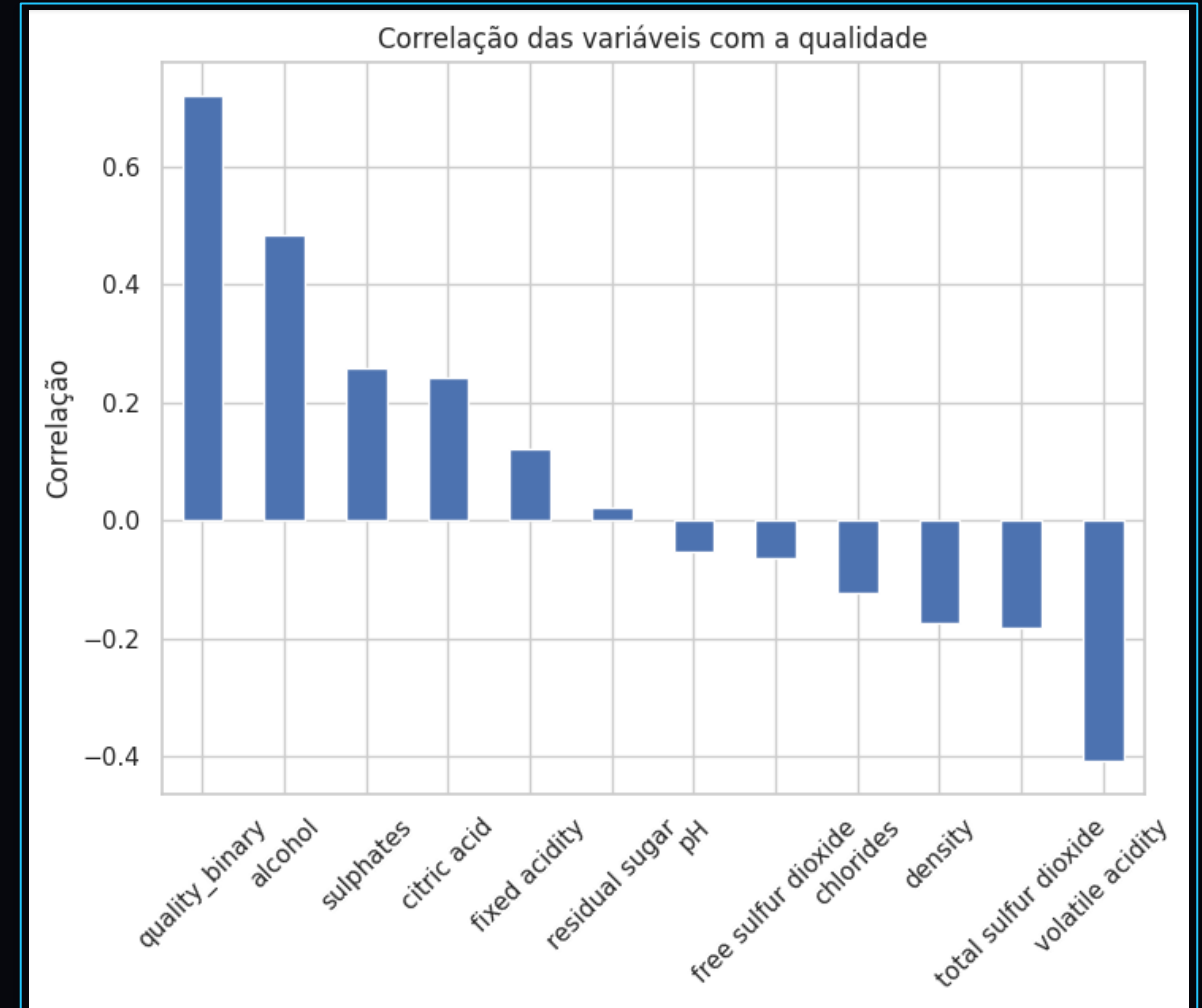
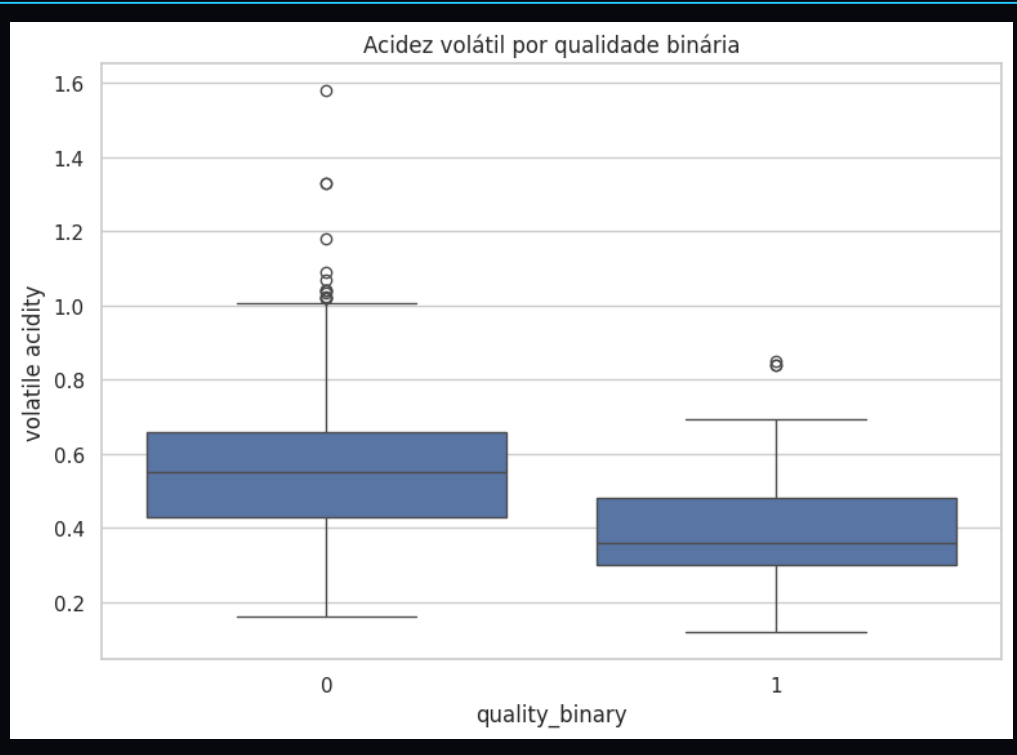
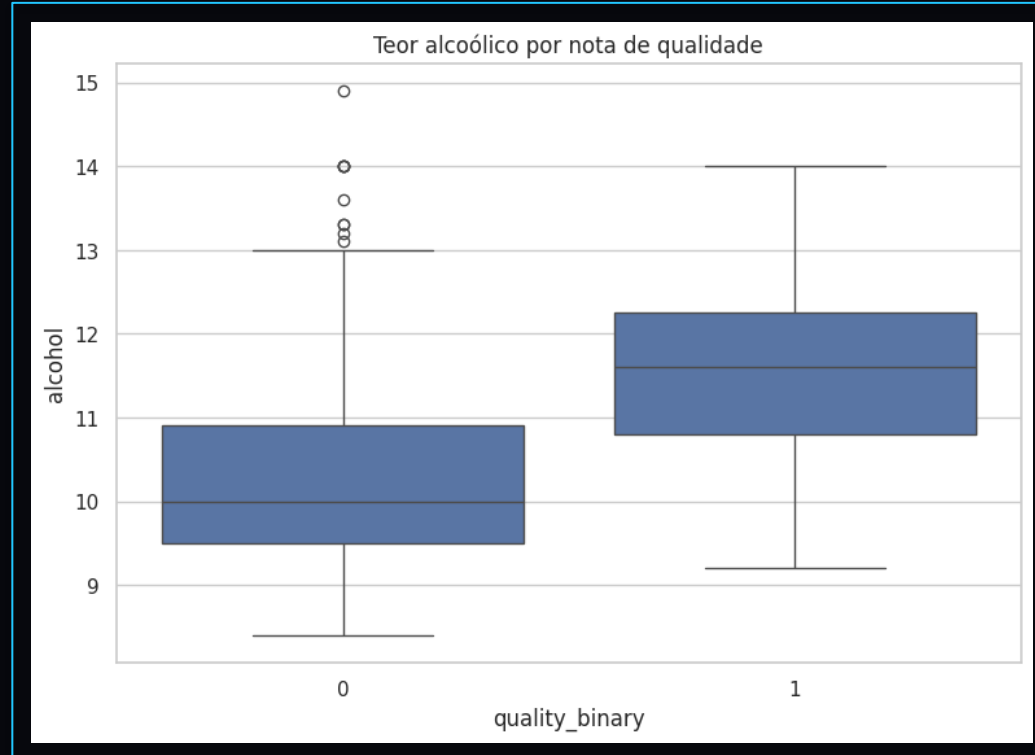


Gráfico de correlação em Barras

Análise Exploratória (EDA)



Boxplot Acidez Volátil x qualidade binária



Boxplot Álcool x qualidade binária

Pré-processamento

- Pré-processamento da base de dados para aplicação de modelos

Quantidade de amostras para treino: 800
Quantidade de amostras para teste: 343

Resultado do código `train_test_split`

70% Treino = 800 amostras
30% Teste = 343 amostras

Padronização das variáveis das amostras (de 0 a 1)

Randomização da base

Treino normalizado											
	alcohol	volatile acidity	fixed acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
1130	0.625000	0.190083	0.254545	0.434211	0.102740	0.093489	0.123077	0.070671	0.339941	0.434783	0.166667
143	0.339286	0.165289	0.454545	0.618421	0.034247	0.073456	0.123077	0.063604	0.505140	0.313043	0.302469
774	0.428571	0.500000	0.181818	0.118421	0.315068	0.175292	0.123077	0.038869	0.475771	0.426087	0.098765
399	0.482143	0.433884	1.000000	0.644737	0.226027	0.138564	0.138462	0.060071	0.960352	0.052174	0.253086
671	0.714286	0.132231	0.336364	0.631579	0.082192	0.135225	0.076923	0.021201	0.294420	0.347826	0.179012
Teste Normalizado:											
	alcohol	volatile acidity	fixed acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
832	0.500000	0.628099	0.172727	0.039474	0.321918	0.111853	0.338462	0.144876	0.414831	0.626087	0.104938
201	0.375000	0.231405	0.390909	0.421053	0.321918	0.125209	0.138462	0.144876	0.662996	0.452174	0.271605
221	0.196429	0.239669	0.518182	0.552632	0.102740	0.335559	0.076923	0.028269	0.685022	0.286957	0.179012
778	0.642857	0.305785	0.254545	0.355263	0.082192	0.098497	0.200000	0.067138	0.279736	0.426087	0.185185
868	0.928571	0.247934	0.045455	0.000000	0.061644	0.053422	0.261538	0.289753	0.110132	0.713043	0.246914

Normalização das variáveis pré-selecionadas

Desenvolvimento dos modelos

- Regressão Logística (Tradicional)
- Random Forest(Mais Complexo)

```
Aplicando Modelo 1 - Regressão Logística

Como primeiro algoritmo foi escolhida a "Regressão Logística". Segundo meus estudos de classificação binária. Como esse modelo é sensível à escala das variáveis, serão utilizados pré-processamento.

from sklearn.linear_model import LogisticRegression

modelo_lr = LogisticRegression( random_state=42,
                                max_iter=1000)

Treinando o modelo 1

modelo_lr.fit(
    x_train_normalizado,
    y_train
)

LogisticRegression
LogisticRegression(max_iter=1000, random_state=42)

y_pred_lr = modelo_lr.predict(
    x_test_normalizado
)

y_prob_lr = modelo_lr.predict_proba(
    x_test_normalizado
)[: ,1]
```

Tela de códigos do Google Colab mostrando a aplicação e treinamento do modelo 1 – Regressão Logística

```
Aplicando Modelo 2 - Random Forest

from sklearn.ensemble import RandomForestClassifier

modelo_rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42)

modelo_rf.fit(
    x_train_normalizado,
    y_train
)

RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)

Previsões do Modelo 2 - Random Forest

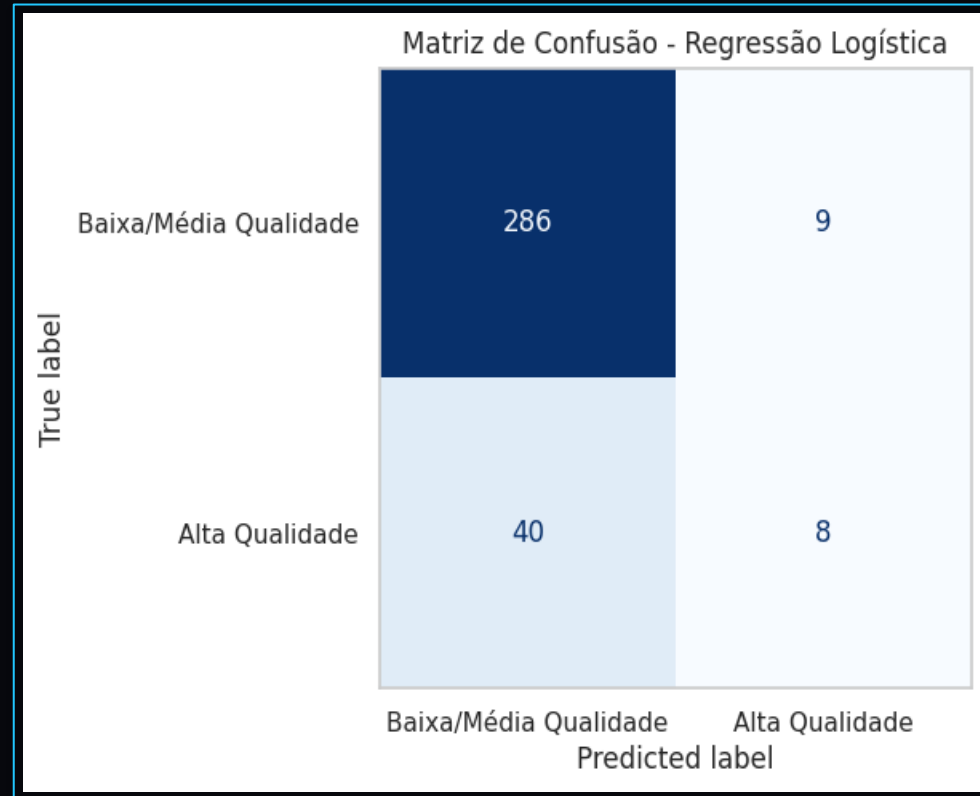
y_pred_rf = modelo_rf.predict(
    x_test_normalizado
)

y_prob_rf = modelo_rf.predict_proba(
    x_test_normalizado
)[: ,1]
```

Tela de códigos do Google Colab mostrando a aplicação e treinamento do modelo 2 – Random Forest

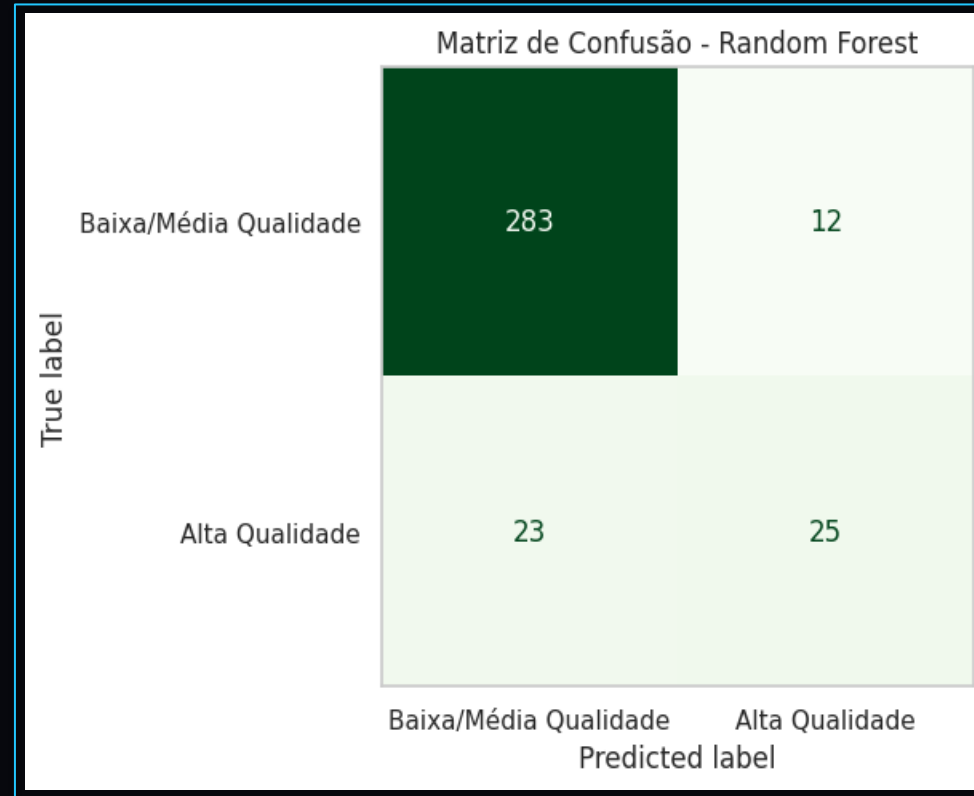
Comparação dos modelos

Regressão Logística



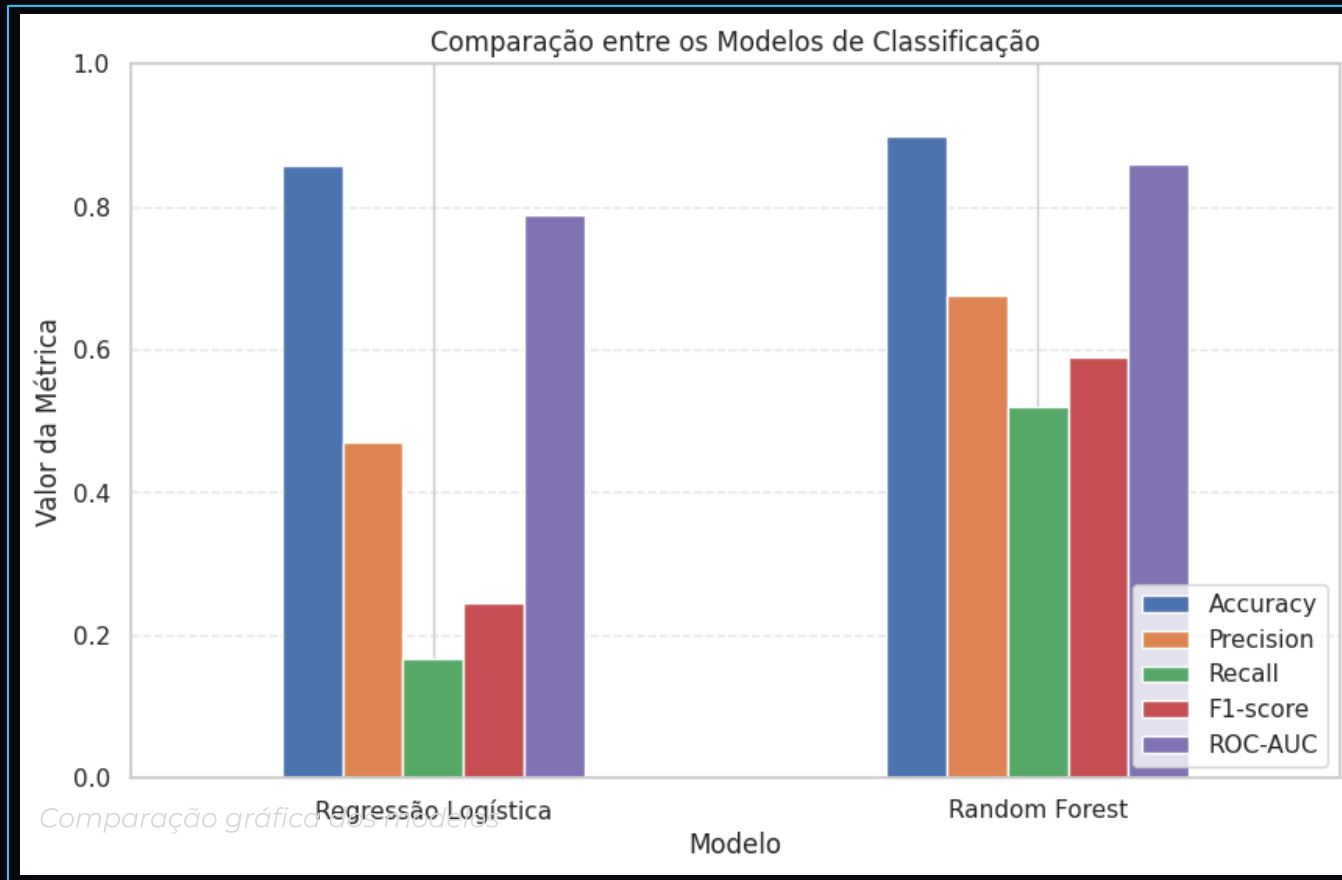
Matriz de confusão – Modelo I (Regressão Logística)

Random Forest



Matriz de confusão – Modelo II (Random Forest)

Comparação dos modelos



- Random Forest: melhor desempenho geral.
- Modelo escolhido para o problema proposto.

Comparação de variáveis em barras

Conclusão

Os resultados demonstraram que as características físico-químicas possuem capacidade preditiva para estimar a qualidade dos vinhos.

Entre as variáveis analisadas, destacaram-se, principalmente, **o teor alcoólico e a acidez volátil**, que apresentaram maior relação com a classificação final.

Comparando os modelos desenvolvidos, o **Random Forest** apresentou o melhor desempenho geral, mostrando-se a alternativa mais adequada para apoiar a previsão da qualidade dos vinhos.